

Practical 2

Aim:-Implementation and Time analysis of linear and binary search algorithm

Linear search;-

Alogrithm :-

A photograph of a piece of paper with the 'Linear Search Algorithm' written in cursive. The text is as follows:

```
Linear Search Algorithm:-
Start
input
input array A
input search
for i=0 to n-1
  if A[i] == search
    print "element found"
  stop
end if
end for
print "element not found"
stop
```

Code:-

```
#include <iostream>
using namespace std;

int main() {
    int n, search;

    cout << "Enter size: ";
    cin >> n;

    int A[n];

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }

    cout << "Enter element to search: ";
    cin >> search;
```

```

for (int i = 0; i < n; i++) {
    if (A[i] == search) {
        cout << "Element Found";
        return 0;
    }
}

cout << "Element Not Found";

return 0;
}

```

Output:-

```

C:\Users\manip\OneDrive\Do
Enter size: 5
Enter array elements: 26 62 6 2 16
Enter element to search: 6
Element Found
Process returned 0 (0x0)   execution time : 23.998 s
Press any key to continue.

```

Time complexity:-

Time Complexity for Linear Search:-

Pseudocode	Time complexity
Start	1 time
input n	1 time
input array A	n times
input search	1 time
for i = 0 to n-1	n times
if A[i] == search	1 time
print "element found"	1 time
stop	1 time
end if	1 time
end for	1 time
print "element not found"	1 time
stop	1 time

Now add all time complexity

$$T(n) = 1 + 1 + n + 1 + n + 1 + 1 + 1 + (n-1)$$

$$T(n) = 2n + 5$$

Avoid constant

$$T(n) = T(n-1) + 1$$

$$T(n) = T(n-n)$$

$$T(n) = T(0)$$

$$T(n) = O(1)$$

Best case:- $O(1)$

element is at the first position

Ex: arr[] = 10 20 30 40 50, key = 10

Average case:- $O(n)$

Element is somewhere in the middle

Ex: arr[] = 10 20 30 40 50, key = 30

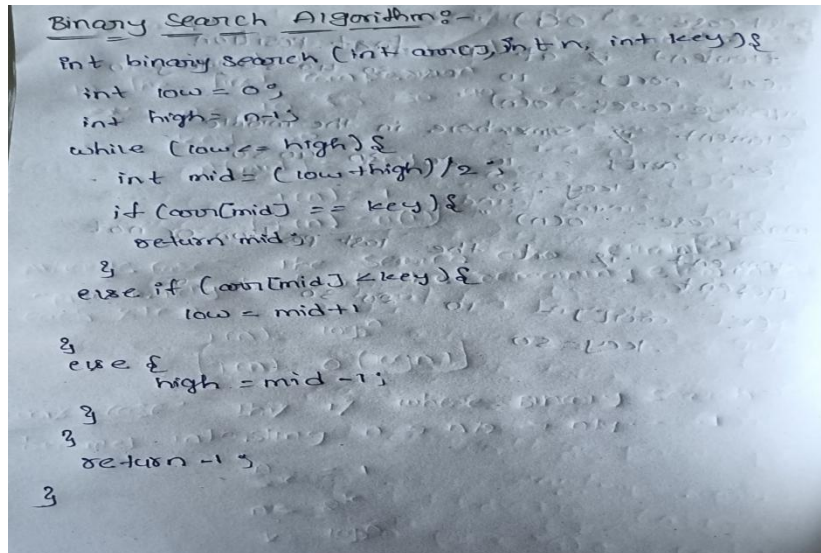
worst case:- $O(n)$

element is at the last position or not present

Ex: arr[] = 10 20 30 40 50, key = 50

Binary search:-

Algorithm :-



Handwritten C++ code for Binary Search Algorithm:

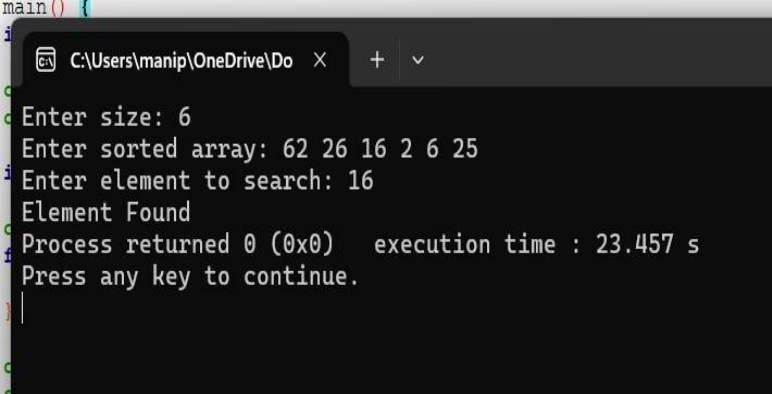
```
Binary Search Algorithm:-  
int binarySearch (int arr[], int n, int key) {  
    int low = 0;  
    int high = n-1;  
    while (low <= high) {  
        int mid = (low+high)/2;  
        if (arr[mid] == key) {  
            return mid;  
        }  
        else if (arr[mid] < key) {  
            low = mid+1;  
        }  
        else {  
            high = mid-1;  
        }  
    }  
    return -1;  
}
```

Code:-

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int n, search;  
  
    cout << "Enter size: ";  
    cin >> n;  
  
    int A[n];  
  
    cout << "Enter sorted array: ";  
    for (int i = 0; i < n; i++) {  
        cin >> A[i];  
    }  
  
    cout << "Enter element to search: ";  
    cin >> search;  
  
    int low = 0;  
    int high = n - 1;  
  
    while (low <= high) {  
        int mid = (low + high) / 2;
```

```
    if (A[mid] == search) {  
        cout << "Element Found";  
        return 0;  
    }  
    else if (search < A[mid]) {  
        high = mid - 1;  
    }  
    else {  
        low = mid + 1;  
    }  
}  
  
cout << "Element Not Found";  
  
return 0;  
}
```

Output:-



The screenshot shows a Windows command prompt window with a dark background. The title bar indicates the file path is C:\Users\manip\OneDrive\Do. The program prompts for input and produces the following output:

```
main() {  
i  
c  
c  
Enter size: 6  
Enter sorted array: 62 26 16 2 6 25  
i  
Enter element to search: 16  
Element Found  
c  
Process returned 0 (0x0)   execution time : 23.457 s  
f  
Press any key to continue.  
}
```

Time complexity :-

Time complexity for Binary Search
Let $T(n)$ = Time to search in array of size n for checking middle element (constant $O(1)$). Search half the array $\rightarrow n/2$
using Recurrence relation :-
So, $T(n) = T(n/2) + O(1)$
 $T(n) = T(n/2) + 1$ (1)
Let $n = n/2$ at each (1)
 $T(n/2) = T(n/4) + 1$ (2)
 $T(n) = T(n/4) + 2$
again $n = n/4$, we get
 $T(n) = T(n/8) + 3$
After some steps we get
 $T(n) = T(n/2^k) + k$
 $\frac{n}{2^k} = 1$ $\therefore$ array become size 1
 $n = 2^k$
Here, add $\log$
 $k = \log_2 n$ $\therefore$ remove constant
 $T(n) = O(\log n)$

Best case :-
The element is found at the middle position in the first comparison
ex: $A = 10 \ 20 \ 30 \ 40 \ 50$
 $key = 30$
 $A[mid] = key$
 $T(n) = 1$
 $T(n) = O(1)$
Average case :- The search also eliminates approximately half of the remaining element at each step
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$
worst case :- This is where binary search becomes interesting $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$
 $\frac{n}{2^k} = 1$
 $n = 2^k$
 $k = \log_2 n$
 $T(n) = \log n$
 $T(n) = O(\log n)$

Conclusion:-

Linear Search and Binary Search were successfully implemented and analyzed. Linear Search is simple and can be used on both sorted and unsorted arrays, but its worst-case time complexity is $O(n)$. Binary Search is more efficient with a sorted array, reducing the search space by half at each step, giving a worst-case time complexity of $O(\log n)$. Therefore, Binary Search is preferred for searching large sorted datasets